```
import java.util.concurrent.atomic.AtomicL

public class Raiyyan_Thread {
    static AtomicLong safe = new AtomicLong()
    static long unsafe = 0;
    Counter (int n, boolean mode) {
        this.n = n;
        this.mode = mode;
    }
    public void run () {
        for (int i=0; i<n; i++) {
            if (mode) safe.incrementAndGet();
            else unsafe++;
            count++;
        }
    }
}
```

```
public static void main(String[] a)throws Except

    int t = Integer.parseInt(a[0]);
    int n = Integer.parseInt(a[1]);

    boolean mode = Boolean.parseBoolean(a[2]);

    safe.set(0);
    unsafe = 0;

    Counter[] c = new Counter[t];

for (int i = 0; i<t; i++) {

    c[i] = new Counter(n, mode);

    c[i].start();

}

long total = 0;

for ( Counter x : c) {

        x.join();

    total += x.count;

}
```

```
long s = mode ? safe.get() : unsafe;
long d = Math.abs(s - total);
System.out.println("Expected : " + (long)(t*n));
System.out.println("Static : " + s);
System.out.println("Non-static total : " + total);
System.out.println("Difference : " + d);
System.out.printf("Difference(%%) : %.2f%%%n",
        total == 0 ? 0 : d * 100.0 / total);
    }
}
```

• Output

Test	Threads	Increments/thread	Expected count	Non-static Total
1	1	1,000	1,000	1,000
2	2	10,000	20,000	20,000
3	5	10,000	50,000	50,000
4	10	50,000	500,000	500,000
5	20	"	1,000,000	1,000,000
6	50	"	2,500,000	2,500,000
7	100	"	5,000,000	5,000,000